CS 301: High-Performance Computing

Assignment 06 - Parallel Interpolation

Assignment 07 – Parallel Interpolation with Particle Mover

Vasu Bhensdadia(202301416)

Darshit Ramani(202301404)

April 19, 2026

Contents

1 Lab6 **3**
- 1.1 Explanation of Implementation Approach . . . 3
- 1.2 Performance Analysis . . . 3
- 1.3 Graphs . . . 4
  - 1.3.1 Lab PC plots . . . 4
  - 1.3.2 Cluster PLot . . . 5
- 1.4 Analysis and Answers . . . 6
  - 1.4.1 Pseudocode of Parallel Implementation . . . 6
  - 1.4.2 Parallel Approach and Race Conditions . . . 6
  - 1.4.3 Graph Observations . . . 6
  - 1.4.4 Parallel Efficiency and Scalability . . . 7
  - 1.4.5 Parallel vs Serial Performance . . . 7
  - 1.4.6 Explanation of Speedup . . . 7
  - 1.4.7 Further Optimization (Unlimited HPC) . . . 7
  - 1.4.8 Load Imbalance and Bottlenecks . . . 7
  - 1.4.9 Alternative Approach . . . 8
  - 1.4.10 Improvements for Performance . . . 8

2 Lab7 **8**
- 2.1 Implementation Approach . . . 8
- 2.2 Performance Analysis . . . 9
- 2.3 Graph . . . 10
  - 2.3.1 Lab PC plots . . . 10
  - 2.3.2 Cluster plots . . . 11
- 2.4 Analysis and Questions . . . 11
  - 2.4.1 Pseudocode . . . 11
  - 2.4.2 Parallel Approach and Race Conditions . . . 12
  - 2.4.3 Graphs . . . 12
  - 2.4.4 Parallel Efficiency and Scalability . . . 12
  - 2.4.5 Comparison with Serial Version . . . 12
  - 2.4.6 Explanation of Observations . . . 13
  - 2.4.7 Further Optimization (Unlimited Resources) . . . 13
  - 2.4.8 Load Imbalance and Bottlenecks . . . 13
  - 2.4.9 Alternative Approach . . . 13
  - 2.4.10 Improved Data Structures / OpenMP Strategies . . . 13
  - 2.4.11 Phase-wise Analysis (Bottleneck) . . . 13

1 Lab6

1.1 Explanation of Implementation Approach

The interpolation problem is parallelized using a particle-based decomposition strategy with OpenMP. Let the total number of particles be N, and the number of threads be P. The workload is divided such that each thread processes approximately:

$$\frac{N}{P}$$

particles independently.

For each particle located at (x, y), the corresponding grid indices (i, j) are computed as:

$$i = \left\lfloor \frac{x}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y}{\Delta y} \right\rfloor$$

The interpolation weights are computed using bilinear interpolation:

$$w_1 = (\Delta x - l_x)(\Delta y - l_y), \quad w_2 = l_y(\Delta x - l_x)$$
$$w_3 = l_x(\Delta y - l_y), \quad w_4 = l_x l_y$$

where:

$$l_x = x - i\Delta x, \quad l_y = y - j\Delta y$$

To avoid race conditions, each thread maintains a private local mesh $M^{(t)}$. Thus, updates are performed as:

$$M^{(t)}[i, j] += w_k$$

After processing all particles, the global mesh is obtained by combining all thread-local meshes:

$$M = \sum_{t=1}^{P} M^{(t)}$$

This merging step is implemented using a critical section to ensure correctness. While this avoids race conditions, it introduces synchronization overhead, which affects scalability.

1.2 Performance Analysis

The performance is evaluated using execution time T_p and speedup S_p for different numbers of threads P.

Speedup is defined as:

$$S_p = \frac{T_1}{T_p}$$

where T_1 is the execution time using one thread.

For the configuration 250×100 with 900000 points, the execution time decreases from approximately 0.28 seconds (1 thread) to 0.06 seconds (8 threads), giving:

$$S_8 \approx \frac{0.28}{0.06} \approx 4.8$$

However, at 16 threads, the speedup reduces slightly to around 4.5, indicating diminishing returns due to overhead.

For larger datasets such as 250×100 with 5 million points:

$$S_{16} \approx 6.8$$

and for 500×200 with 20 million points:

$$S_{16} \approx 8.3$$

This shows that larger workloads achieve better parallel efficiency.
Parallel efficiency is defined as:

$$E_p = \frac{S_p}{P}$$

For the 900000 case:

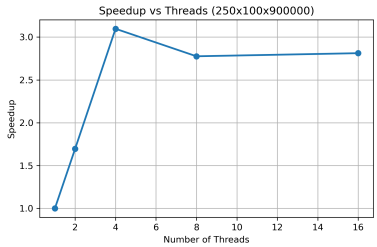
$$E_8 \approx \frac{4.8}{8} = 0.6, \quad E_{16} \approx \frac{4.5}{16} \approx 0.28$$

Overall, the observed performance follows:

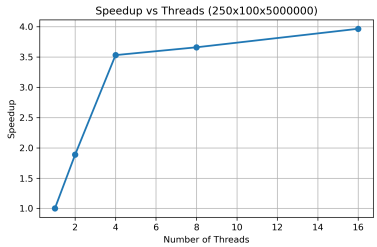
- Speedup increases with threads initially.
- Beyond 8 threads, gains reduce due to synchronization overhead.
- Larger datasets scale better because computation dominates communication and synchronization costs.

1.3 Graphs

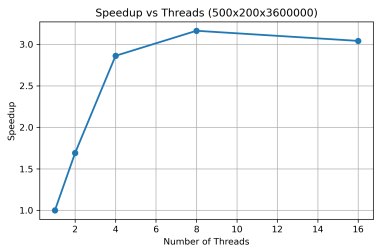
1.3.1 Lab PC plots



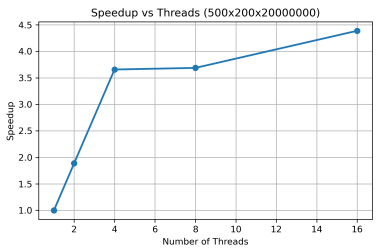
(a) 250x100x900000



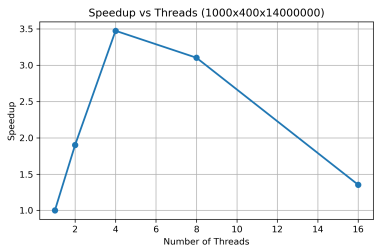
(b) 250x100x5000000



(c) 500x200x3600000

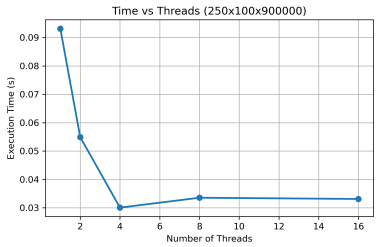


(d) 500x200x20000000

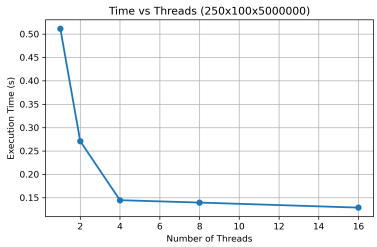


(e) 1000x400x14000000

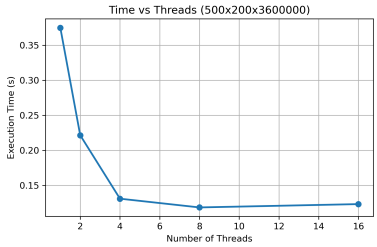
Figure 1: Speedup vs Threads for all configurations in Lab PC



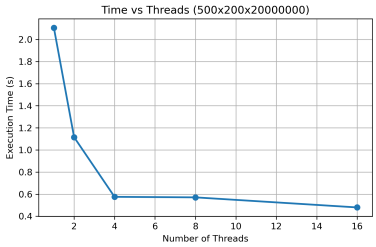
(a) 250x100x900000



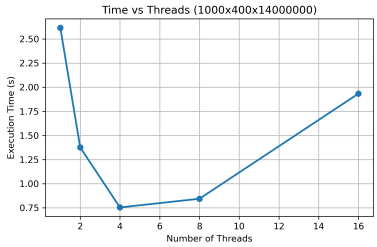
(b) 250x100x5000000



(c) 500x200x3600000



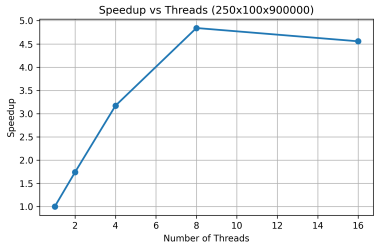
(d) 500x200x20000000



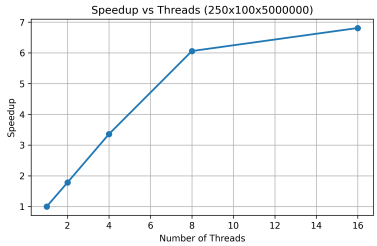
(e) 1000x400x14000000

Figure 2: Execution Time vs Threads for all configurations in Lab PC

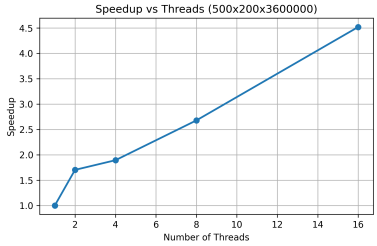
1.3.2 Cluster PLot



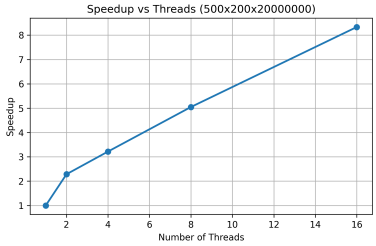
(a) 250x100x900000



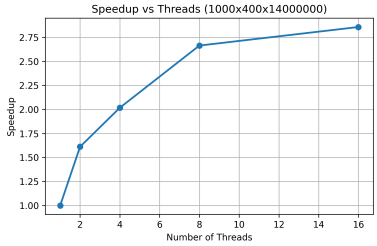
(b) 250x100x5000000



(c) 500x200x3600000

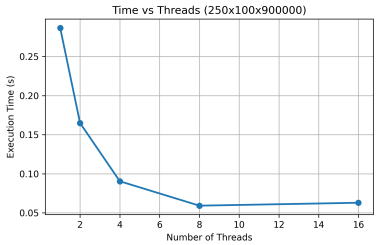


(d) 500x200x20000000

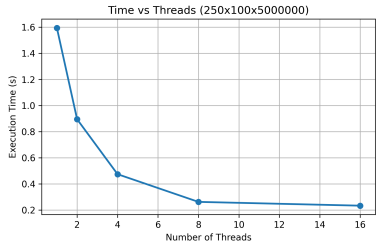


(e) 1000x400x14000000

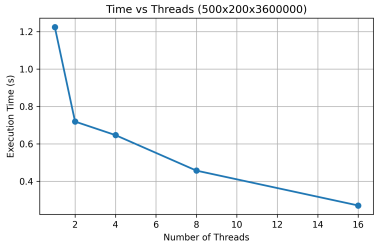
Figure 3: Speedup vs Threads for all configurations in Cluster



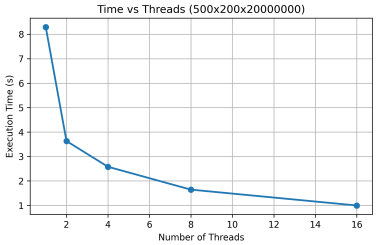
(a) 250x100x900000



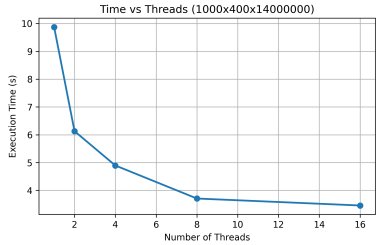
(b) 250x100x5000000



(c) 500x200x3600000



(d) 500x200x20000000



(e) 1000x400x14000000

Figure 4: Execution Time vs Threads for all configurations

1.4 Analysis and Answers

1.4.1 Pseudocode of Parallel Implementation

The implementation follows a particle-based parallelization approach. Each thread processes a subset of particles and updates a private local mesh. Finally, all local meshes are merged into a global mesh.

Pseudocode:

```
for each thread:
    create local_mesh

parallel for each particle:
    find grid cell
    compute weights
    update local_mesh

merge all local_mesh into global mesh
```

1.4.2 Parallel Approach and Race Conditions

The implementation uses particle decomposition, where particles are divided among threads. To avoid race conditions, each thread maintains a private local mesh. Since threads do not directly update the global mesh, conflicts are avoided. A critical section is used to combine local meshes at the end.

1.4.3 Graph Observations

From the plots:

250x100x900000: Speedup increases up to 8 threads (around 4.8) but slightly decreases at 16 threads.

250x100x5000000: Speedup increases consistently up to 16 threads (around 6.8).

500x200x3600000: Speedup grows slowly and reaches around 4.5 at 16 threads.

500x200x20000000: Speedup increases steadily and reaches around 8.3 at 16 threads.

1000x400x14000000: Speedup is limited and reaches only around 2.8 at 16 threads.

Execution time decreases in all cases, but improvement slows after 8 threads in smaller cases.

1.4.4 Parallel Efficiency and Scalability

Efficiency decreases as the number of threads increases.

For small workloads (900k points), efficiency drops quickly after 8 threads, showing poor scalability.

For larger workloads (5M and 20M points), efficiency remains higher and scaling is better.

Thus, scalability improves with problem size.

1.4.5 Parallel vs Serial Performance

Parallel execution is significantly faster than serial execution.

Maximum speedup observed:

- $\sim$4.8 for 900k points
- $\sim$6.8 for 5M points
- $\sim$4.5 for 3.6M points
- $\sim$8.3 for 20M points (best case)
- $\sim$2.8 for 14M points

1.4.6 Explanation of Speedup

Speedup increases initially because work is divided among threads.

However:

- After 8 threads, gains reduce or even drop (seen in 900k case)
- Critical section merging slows execution
- Memory access becomes a bottleneck

Larger datasets show better speedup because computation dominates overhead.

1.4.7 Further Optimization (Unlimited HPC)

If unlimited resources are available:

- Remove critical section using parallel reduction
- Use GPU acceleration
- Optimize memory layout for cache efficiency
- Use domain-based parallelization instead of particle-based

1.4.8 Load Imbalance and Bottlenecks

Load imbalance is minimal due to static scheduling.

Main bottlenecks:

- Critical section during merging
- Memory bandwidth limitation
- Large local mesh allocations

1.4.9 Alternative Approach

An alternative is grid-based parallelization:

- Assign threads to grid cells instead of particles
- Avoid merging overhead

Another option:

- Use atomic updates instead of critical section

1.4.10 Improvements for Performance

Possible improvements:

- Replace critical section with reduction
- Use better memory layout to reduce cache misses
- Use dynamic scheduling for uneven workloads
- Reduce memory allocation overhead

Large datasets already show good scaling, so optimization should focus on removing synchronization overhead.

2 Lab7

2.1 Implementation Approach

The implementation models a particle-to-grid and grid-to-particle interaction over a structured domain. A set of particles (x_i, y_i) is mapped onto a grid using bilinear interpolation, followed by a reverse mapping that updates particle positions.

For each particle, its enclosing grid cell is identified using:

$$i = \left\lfloor \frac{x_i}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y_i}{\Delta y} \right\rfloor$$

The contribution to the four neighboring grid points is computed using local distances:

$$l_x = x_i - X_i, \quad l_y = y_i - Y_j$$

and weights:

$$w_1 = (\Delta x - l_x)(\Delta y - l_y), \quad w_2 = l_y(\Delta x - l_x)$$
$$w_3 = l_x(\Delta y - l_y), \quad w_4 = l_x l_y$$

The grid values are accumulated using these weights. To avoid race conditions in parallel execution, each thread maintains a private copy of the grid, and a final reduction is performed:

$$F = \sum_{t=1}^{T} F^{(t)}$$

The grid is then normalized to $[-1, 1]$ using global min-max reduction. Reverse interpolation is applied where each particle samples from the grid using the same weights:

$$F_i = \sum_{k=1}^{4} w_k F_k$$

Particle positions are updated as:

$$x_i^{new} = x_i + F_i \Delta x, \quad y_i^{new} = y_i + F_i \Delta y$$

Particles leaving the domain are marked inactive. Finally, grid values are restored via denormalization. This pipeline is repeated for each iteration, where interpolation is write-intensive and mover is read-dominant, enabling efficient parallelization.

2.2 Performance Analysis

The total execution time is dominated by interpolation and mover phases. From the experimental results, the execution time decreases as the number of threads increases, but not linearly.

For the smallest case (250×100, 0.9M points), execution time reduces from:

$$T(1) = 0.2887 \text{ s} \rightarrow T(16) \approx 0.0904 \text{ s}$$

giving speedup:

$$S(16) \approx 3.19$$

For larger input (250×100, 5M points):

$$T(1) = 1.589 \text{ s}, \quad T(16) = 0.391 \text{ s} \Rightarrow S(16) \approx 4.06$$

For (500×200, 3.6M points):

$$T(1) = 3.000 \text{ s}, \quad T(16) = 0.447 \text{ s} \Rightarrow S(16) \approx 6.70$$

For the largest case (500×200, 20M points):

$$T(1) = 8.230 \text{ s}, \quad T(16) = 1.782 \text{ s} \Rightarrow S(16) \approx 4.61$$

For (1000×400, 14M points):

$$T(1) = 6.240 \text{ s}, \quad T(16) = 3.496 \text{ s} \Rightarrow S(16) \approx 1.78$$

Efficiency is defined as:

$$E(p) = \frac{S(p)}{p}$$

For example, for the 3.6M case:

$$E(16) = \frac{6.70}{16} \approx 0.42$$

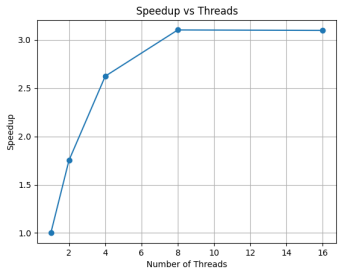
The results show that performance improves with problem size, as larger workloads better amortize parallel overhead. However, scalability is limited due to memory bandwidth constraints and reduction overhead in interpolation.

It is also observed that performance does not monotonically improve with threads. For example, in some cases (e.g., 20M points), 2 threads perform worse than 1 thread, indicating parallel overhead dominates at low thread counts.

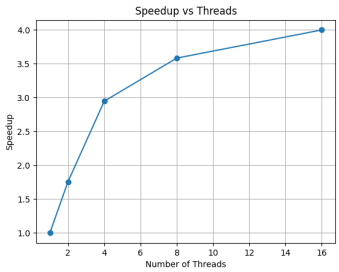
Interpolation acts as the primary bottleneck due to concurrent updates to shared grid regions, while the mover phase scales better as it involves independent particle updates. At higher thread counts, cache contention and memory access latency limit further speedup.

2.3 Graph

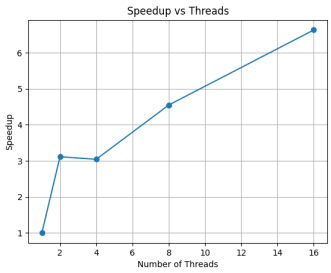
2.3.1 Lab PC plots



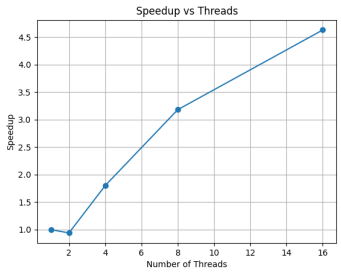
(a) 250x100x900000



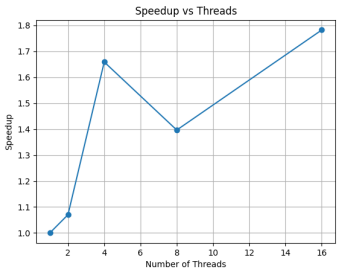
(b) 250x100x5000000



(c) 500x200x3600000

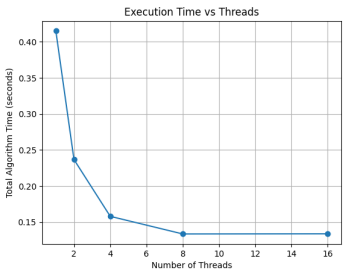


(d) 500x200x20000000

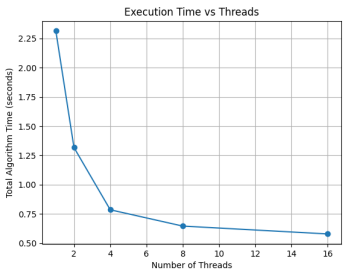


(e) 1000x400x14000000

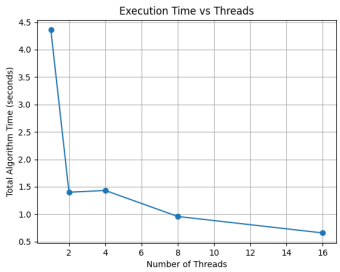
Figure 5: Speedup vs Threads for all configurations in Lab PC



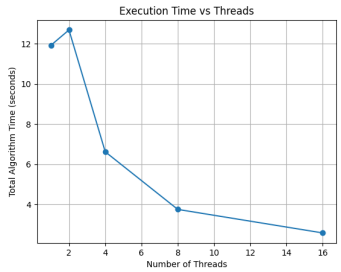
(a) 250x100x900000



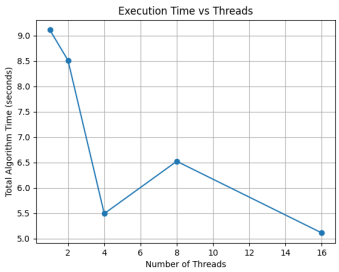
(b) 250x100x5000000



(c) 500x200x3600000



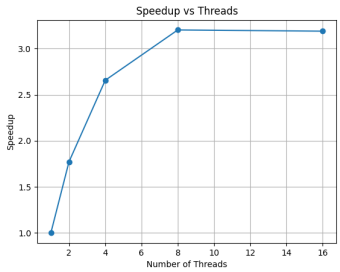
(d) 500x200x20000000



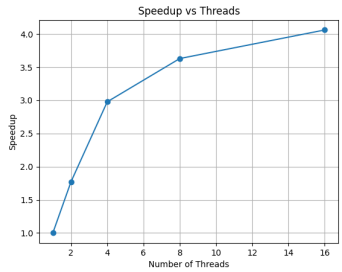
(e) 1000x400x14000000

Figure 6: Execution Time vs Threads for all configurations in Lab PC

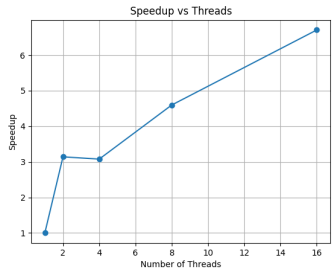
2.3.2 Cluster plots



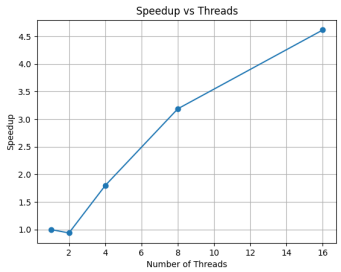
(a) 250x100x900000



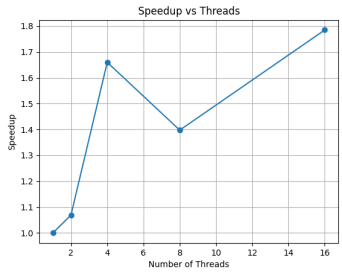
(b) 250x100x5000000



(c) 500x200x3600000

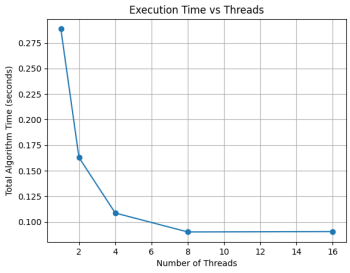


(d) 500x200x20000000

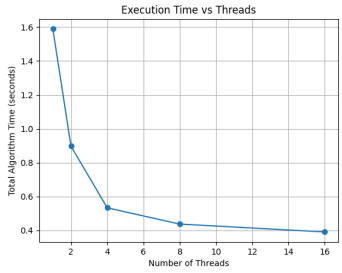


(e) 1000x400x14000000

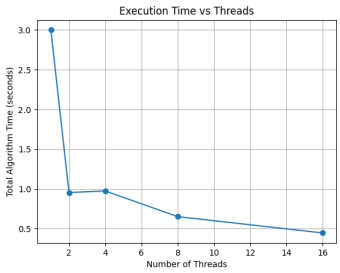
Figure 7: Speedup vs Threads for all configurations in cluster



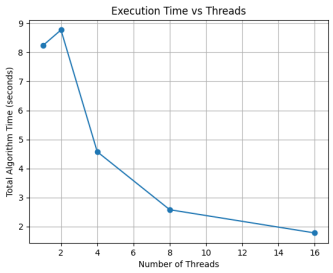
(a) 250x100x900000



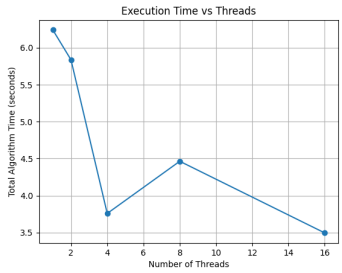
(b) 250x100x5000000



(c) 500x200x3600000



(d) 500x200x20000000



(e) 1000x400x14000000

Figure 8: Execution Time vs Threads for all configurations in cluster

2.4 Analysis and Questions

2.4.1 Pseudocode

```
for iter = 1 to Maxiter:
    parallel initialize mesh = 0
```

```
parallel:
    each thread uses local mesh
    for each particle:
        compute (i,j)
        compute weights
        update local mesh

merge all local meshes → global mesh

parallel reduction → find min, max
parallel normalize mesh

parallel for each particle:
    interpolate from mesh
    update position
    mark void

parallel denormalize mesh
```

2.4.2 Parallel Approach and Race Conditions

Interpolation causes write conflicts since multiple particles update the same grid. This is handled using thread-private grids:

$$F = \sum_{t=1}^{T} F^{(t)}$$

This avoids atomic operations and ensures deterministic results. The mover phase is naturally parallel since particles are independent.

2.4.3 Graphs

Speedup vs threads and execution time vs threads are plotted for all configurations as required in the assignment :contentReference[oaicite:2]index=2.

2.4.4 Parallel Efficiency and Scalability

Efficiency is defined as:

$$E(p) = \frac{T(1)}{p \cdot T(p)}$$

For example (500×200, 3.6M points):

$$S(16) = \frac{3.0007}{0.4474} \approx 6.70, \quad E(16) \approx 0.42$$

Efficiency decreases with threads due to memory contention and reduction overhead. Larger problems show better scalability.

2.4.5 Comparison with Serial Version

For different cases:

$$\text{Case 1: } S(16) \approx \frac{0.2887}{0.0904} \approx 3.19$$

$$\text{Case 2: } S(16) \approx \frac{1.589}{0.391} \approx 4.06$$

$$\text{Case 3: } S(16) \approx \frac{3.0007}{0.4474} \approx 6.70$$

$$\text{Case 4: } S(16) \approx \frac{8.230}{1.782} \approx 4.61$$

$$\text{Case 5: } S(16) \approx \frac{6.240}{3.496} \approx 1.78$$

Maximum speedup observed is:

$$S_{max} \approx 6.70$$

2.4.6 Explanation of Observations

Speedup increases with problem size since computation dominates overhead. However, scalability is limited by memory bandwidth, cache contention, and reduction overhead. In some cases, lower thread counts perform worse due to parallel overhead dominating useful work.

2.4.7 Further Optimization (Unlimited Resources)

Performance can be improved using GPU acceleration, distributed memory (MPI), and better data locality through domain decomposition. Vectorization and optimized memory layout can further reduce execution time.

2.4.8 Load Imbalance and Bottlenecks

Load imbalance occurs as particles become inactive over iterations, leading to uneven work distribution. The main bottleneck is interpolation due to heavy memory writes and contention, while other phases are relatively lightweight.

2.4.9 Alternative Approach

Using domain decomposition can reduce contention by assigning grid regions to threads. Atomic-based approaches are possible but introduce higher synchronization overhead.

2.4.10 Improved Data Structures / OpenMP Strategies

Using contiguous memory layout improves cache usage, while dynamic scheduling helps balance workload. Padding can reduce false sharing, and optimized reductions can lower synchronization cost.

2.4.11 Phase-wise Analysis (Bottleneck)

Interpolation dominates execution time due to multiple memory updates per particle. The mover phase scales better as it is read-heavy and independent. Hence, interpolation is the primary bottleneck.